

NETWORK ON CHIP (NoC)

Exam Notes - AHA Unit 3

1. Introduction & Motivation

- By 2005, single-core performance saturated due to power dissipation limits and diminishing returns from ILP.
- Caches became large multi-banked structures; timing dominated by distance to farthest banks.
- Shift to multicore/manycore processors: 10-20 cores + equivalent cache banks + memory controllers + accelerators.
- All elements (cores, cache banks, memory controllers) need to communicate via messages.
- On-chip network (NoC) routes messages from senders to receivers.
- Unlike internet, NoC cannot drop messages — needs complex protocols for reliable delivery.

1.1 Key Problems in NoC

- **Deadlock:** Circular loop of dependences among packets preventing progress.
- **Starvation:** A packet cannot reach destination because other packets cause it to stall.
- **Livelock:** Packet moves round-and-round in circles, never reaching destination.

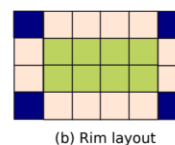
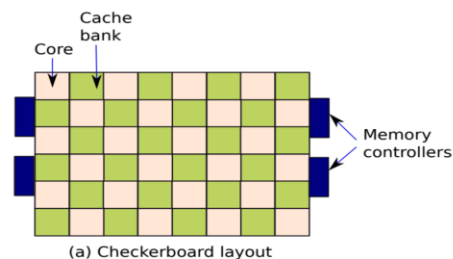
1.2 Multicore Layouts

- Cores & cache banks arranged as chessboard OR cores on rim with cache banks in centre.
- Memory controllers always on periphery.
- Accelerators: centre (compute-heavy) or periphery (memory-heavy).

Architectures for Hardware Acceleration

Network on Chips

- The typical layout of a manycore processor die looks like one of the sub-figures in Figure. The cores and cache banks are either organised as a chess board, or the cores are on the rim and the cache banks are in the centre. The memory controllers are always located on the periphery.
- Dedicated accelerators can either be in the centre if their primary job is to perform computations or can be towards the periphery if they need to communicate a lot with memory.



Different kinds of multicore layouts

Fig: Different kinds of multicore layouts

2. Nodes and Links

- NoC modelled as a graph: nodes (routers) + edges (communication channels/links).
- Node: generic communication unit attached to core/cache bank/memory controller.
- Router: realization of a node. One router per tile (group of cores + cache banks).
- Message: set of bytes sent atomically from sender to receiver (all-or-nothing delivery).
- Link: physical channel (copper wires) between two routers.

Architectures for Hardware Acceleration

Network on Chips



A router is a generic communication unit in an NoC. Every component that wishes to communicate using the NoC needs to have access to a router that sends and receives messages on its behalf. In addition, in modern networks a message is sent from a source to a destination by passing it from router to router. The routers cooperate and coordinate among themselves to deliver the message at the final destination.

Fig: NoC node and link structure

Architectures for Hardware Acceleration

Network on Chips



- *A simple bus is not enough for connecting multiple cores and cache banks in a modern chip – there is a lot of contention.*
- *We need to have complex interconnections between the communicating routers.*
- *It is thus necessary to create multiple metal layers, where the layers close to the silicon are for local wires, and the layers farther away contain longer interconnects known as global wires.*
- *A connection from a given source to a destination might have segments in different layers.*
- *Copper wires across different layers are connected by trans-silicon vias (TSVs).*

Fig: NoC topology example

3. Network Topologies

3.1 Mesh

- Simple 2D matrix; adjacent nodes on same row/column connected by links.

- Uses rectilinear (horizontal/vertical) links — easy to fabricate in VLSI.
- Diameter of $N \times N$ mesh = $2N - 2$ (distance between diagonally opposite corners).
- Popular in on-chip networks due to fabrication simplicity.

Architectures for Hardware Acceleration

Network on Chips: Mesh and Torus



- Figure shows the mesh topology. A mesh is a simple 2D matrix, where we connect two adjacent nodes on the same row or the same column with a link.
- The mesh is a very popular structure in on-chip networks because it is very easy to create. It uses rectilinear links that are either horizontal or vertical.
- Fabricating such structures is very easy in modern VLSI processes. In comparison, it is fairly difficult to fabricate wires that are at oblique angles (neither vertical nor horizontal).

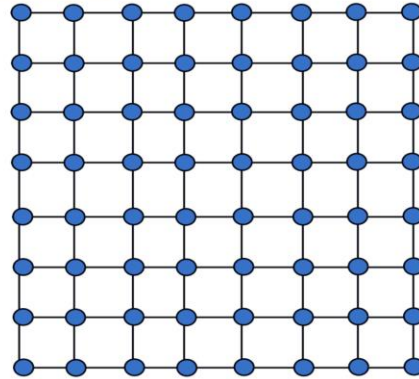


Figure 8.4: The mesh topology

Fig: Mesh topology

3.2 Torus

- Mesh + wraparound links between ends of each row and column.
- Diameter reduced to N (for $N \times N$ torus, assuming N is even) — roughly half of mesh.
- Problem: introduces long wires spanning chip length $\rightarrow$ large delays.
- Solution: Folded Torus — node connected to node 2 columns away instead of adjacent.
- Folded torus eliminates long wires but increases adjacent-node delay by factor 2.

Architectures for Hardware Acceleration

Network on Chips



- The only addition is the long wires between the ends of each row and the ends of each column. They effectively reduce the diameter.
- In this case, the nodes that are the farthest apart are the centre and any of the corners. The diameter is $N/2 + N/2 (= N)$.
- Note that the assumption is that N is even. We have thus reduced the effective diameter roughly by a factor of 2.
- However, we have also increased the number of wires and introduced a few very long wires that span the length of the chip. Such long wires will have very large delays and may not give us any significant advantage.
- Thus, most designs use the folded torus design shown in Figure 8.6.

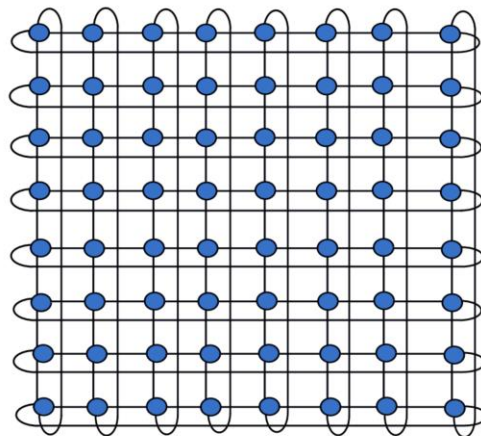


Figure 8.5: The torus topology

Fig: Torus topology

- Note that the design shown in Figure 8.6 is equivalent to the torus shown in Figure 8.5. However, the connections have been made in a different manner.
- A node in a row is not connected to the node that is directly adjacent to it (in the next column). Instead it is connected to the node (on the same row) that is two columns away.
- We have a similar connection pattern for the nodes in each column. Even though such designs increase the timing delay between adjacent nodes by a factor of 2; however, they eliminate long wires completely

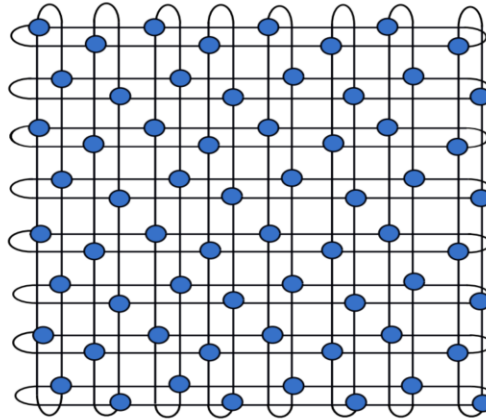


Figure 8.6: The folded torus topology

Fig: Folded Torus design

3.3 High-Radix Networks

- Mesh/Torus = low-radix (2-4 connections per node).
- High-radix: more links per node $\rightarrow$ lower diameter + higher path diversity.
- Path Diversity: number of different paths between source-destination pair.
- Better reaction to congestion; used in cluster computers.
- Flattened networks: high-radix networks rearranged for 2D VLSI fabrication.

3.4 Hypercube

- Recursive construction: H_0 = single node; H_n = two copies of H_{n-1} connected together.
- Nodes labelled with $\log_2(N)$ -bit binary numbers.
- Each link traversal flips exactly one bit in the label.
- Properties (N nodes, N = power of 2):
 - Each node connected to $\log_2(N)$ neighbors
 - Diameter = $\log_2(N)$
 - Routing: flip differing bits one at a time (Hamming distance)

- Figure 8.7 shows the hypercube topology. A hypercube refers to a family of network topologies that are constructed recursively.
- An order 0 hypercube H_0 is a single node. To construct an order 1 hypercube H_1 we take two copies of order 0 hypercubes and connect them together (see Figure 8.7(b)).
- Similarly, to create H_2 we take two copies of H_1 and connect the corresponding nodes together. Observe that we can number the nodes as binary numbers.
- For example, the nodes in H_2 can be numbered 00, 01, 11, and 10 respectively.

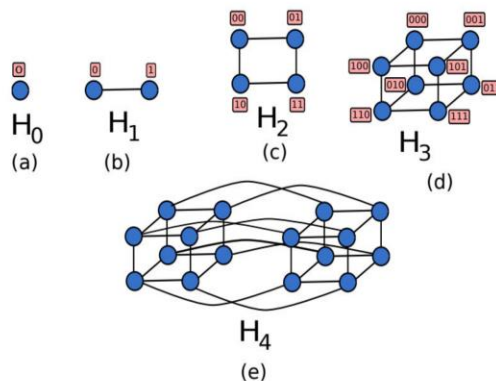
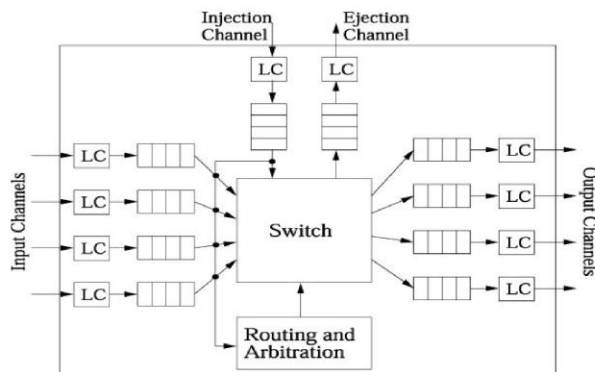


Figure 8.7: The hypercube topology

Fig 8.7: Hypercube topology (H_0 to H_3)

3.5 Router Architecture Overview

- Major components: Buffers, Switch, Routing & Arbitration Unit, Link Controller.
- Switch connects input buffers to output buffers.
- Routing & arbitration unit implements connection algorithm.



- The major components of any routing node that is, buffers, switch, routing and arbitration unit, and link controller.
- The switch connects the input buffers to the output buffers, while the routing and arbitration unit implements the algorithm that dictates these connections.

Fig: Router architecture components

3.6 Clos Network

- Three-layer network with parameters: n , m , r .
- Ingress layer: r switches of $[n \times m]$ (n inputs, m outputs).
- Middle layer: m switches of $[r \times r]$.
- Egress layer: r switches of $[m \times n]$.

- Total: nr inputs, nr outputs.
- **Key Properties:**
 1. If $m \geq n$: can always connect unused ingress-egress terminal by rearranging connections.
 2. If $m \geq 2n-1$: can connect without any rearrangement (strictly non-blocking).
- Folded Clos: input and output terminals connected to same set of routers.

Architectures for Hardware Acceleration

Network on Chips: Clos Network



- In Figure 8.8 we show a Clos network. Typical three-layer Clos networks are traditionally described using three parameters: n , m , and r .
- The first (leftmost) layer or the ingress layer contains r $[n \times m]$ switches. An $n \times m$ switch has n inputs and m outputs. It implements an all-to-all connection where any input can be connected to any output.

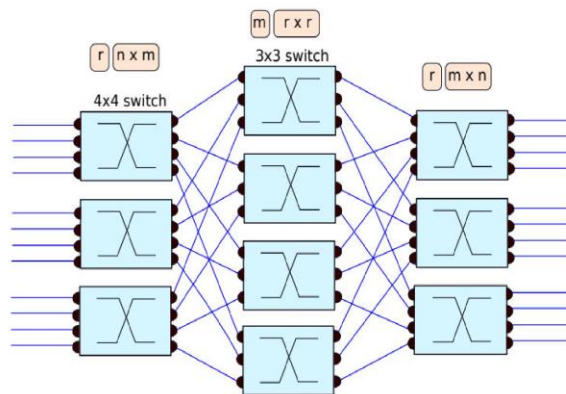


Figure 8.8: The Clos network

Fig 8.8: Clos Network ($n=4$, $m=4$, $r=3$)

3.7 Butterfly Network

- N inputs, N outputs, $\log_2(N)$ layers of 2×2 switches.
- Each layer progressively narrows destination set by half.
- Benes network: special case of Clos with $m=n=2$, uses only 2×2 switches.
- Folded Butterfly: input and output nodes are the same.
- Lacks path diversity (single path per source-dest pair) compared to Clos.
- Links: $N + N \cdot \log_2(N)$ links vs $2N$ for torus.
- Diameter: $\log_2(N) + 1$ (much better than torus diameter $\approx N$).

- Let us look at a similar network called a Butterfly network that uses low-radix switches. High performance implementations of Butterfly networks can use high-radix switches;
- The design of a basic Butterfly network is shown in Figure 8.9. The network has N input and N output terminals, and $\log_2(N)$ layers of 2×2 switches.

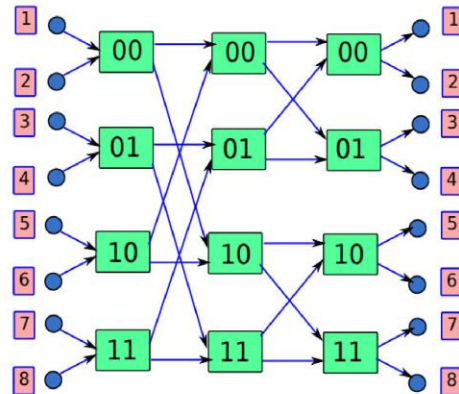


Figure 8.9: The Butterfly network

Fig 8.9: Butterfly network

4. Message Transmission Concepts

- **Hierarchy:** Message $\rightarrow$ divided into Packets $\rightarrow$ divided into Flits $\rightarrow$ transmitted as Phits.
- Message: sequence of bytes forming one logical unit.
- Packet: fixed-size unit; all bits flow along same path. Not dropped in NoC.
- Flit (Flow Control Digit): basic unit for storage/transmission in routers (8 or 16 bytes).
- **Flit Types:**
 - Head flit: contains destination router ID, route info
 - Body flits: data payload
 - Tail flit: last flit in packet
- All flits of same packet follow same route (determined by head flit).
- Phit (Physical Digit): group of bits transferred in one cycle on physical wires.
- Example: 64-bit flit on 32-wire link = 2 phits per flit.
- Flits cannot be split across routers. Phits are invisible to routers.

Message A message is a stream of bytes that makes sense at the level of the application.

Packet A message is divided into a sequence of packets where the NoC treats a packet as a consolidated unit. All the bytes in the packet follow the same path.

Flits Packets are further subdivided into flits of the same size. All the flits in the packet follow the same route. Additionally, each router has buffers, where each entry can store a single flit.

Phits Due to limitations in the signalling technology, it is not possible to have very wide links between two neighbouring routers. We thus have narrow links and send a flit over multiple clock cycles. We thus divide a flit into multiple phits, where a phit (physical digit) represents the set of bits that are sent in a single clock cycle.

Fig: Message → Packet → Flit → Phit hierarchy

1. We have discussed messages, packets, flits, and phits. They need to be sent from a sender to a destination through a sequence of routers.
2. Each router needs to read in the message, a flit at a time, temporarily store the flits, and forward them on its outgoing links to a neighbouring router.

Fig: Transmission structure detail

5. Flow Control

- Flow control = process of allocating/deallocating buffer space along a path.
- Three granularities: message-level, packet-level, flit-level.
- Channel: unidirectional link between two adjoining nodes.
- Key constraint: never allowed to drop flits/packets in NoC.

5.1 Credit-Based Flow Control

- Router A maintains credit count = estimate of free buffers at downstream router B.

- Initially: credits = number of buffers in B.
- When A sends a flit: decrement credit count.
- When B frees a buffer: sends credit message to A → A increments credit.
- If credits = 0: A stops sending (stalls).

Architectures for Hardware Acceleration

Network on Chips: Credit based Flow Control



- Now if we look at B, it sends a message to increment the credit count to A whenever it frees a buffer for the A → B link.
- If A runs out of credits, then it stops sending messages. Here, the overhead is the number of additional credit messages: one per freed buffer. Figure 8.10 shows an example of flow control using credits.

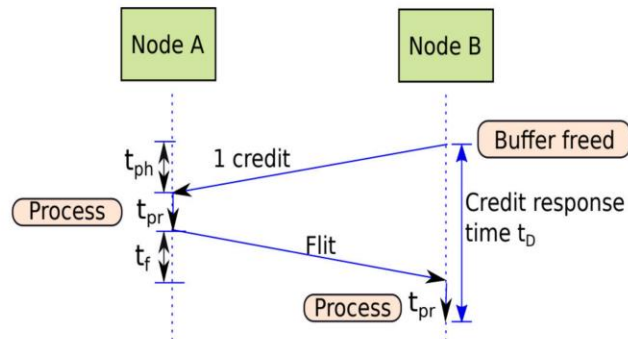


Figure 8.10: Credit based flow control

Fig 8.10: Credit-based flow control example

- Credit Round Trip Delay: $t_D = t_f + t_{ph} + 2*t_{pr}$
- where t_f = time to send a flit, t_{ph} = phit traversal time, t_{pr} = processing time
- To avoid stalls: B needs at least t_D/t_f buffers.
- If starting credits $\geq t_D/t_f$, A will never run out of credits.
- Disadvantage: one credit message per freed buffer → high overhead.

5.2 On-Off Flow Control

- Two thresholds: N_{OFF} (off threshold) and N_{ON} (on threshold).
- When free buffers at B = N_{OFF} : B sends OFF message to A → A stops.
- When free buffers at B = N_{ON} : B sends ON message to A → A resumes.
- $N_{OFF} \geq t_D/t_f$ (must buffer all in-flight flits after OFF sent).
- $N_{ON} > N_{OFF}$ to reduce frequency of ON/OFF messages.
- Shadow of ON message: interval between sending ON and receiving first flit.
- If N_{ON} = total buffers, B is idle during shadow → suboptimal.
- Trade-off: reduce ON/OFF messages vs. minimize idle time.

- On these lines, let us propose a protocol called on-off flow control, where we propose a set of rules that A can use to decide whether it can send a flit to B.
- Let us create two thresholds: NON (on threshold) and NOFF (off threshold). If the number of free buffers in B becomes equal to NOFF, then it sends a message to A to stop sending.
- Once A receives this message, it stops sending flits. On the flip side, when the number of free buffers becomes equal to NON, B sends a message to A to start sending flits.
- Figure 8.11 shows an example of flit transmission using on-off flow control.

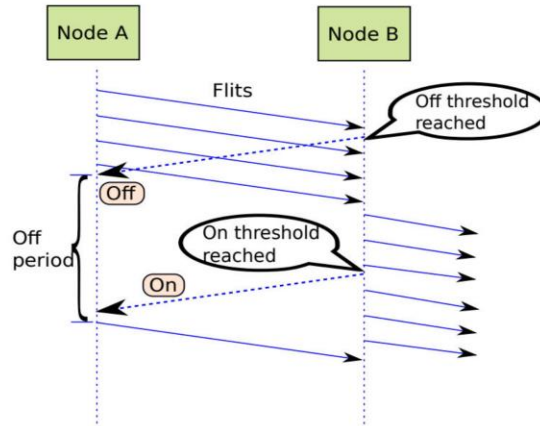


Figure 8.11: On-off flow control

Fig 8.11: On-Off flow control

- Consider the communication shown in Figure 8.12. Let us assume that NON is equal to the total number of buffers at B. If B has N buffers, we are assuming that $N = N_{ON}$.
- Additionally, let the interval between sending an on message and receiving a flit because of it, be referred to as the shadow of the on message.
- Since N_{ON} is assumed to be equal to the total number of buffers, which means that we send an on message when all the buffers are free, B will be idle in the shadow of the on message.
- It will not have any flits to send to any downstream router. This is a suboptimal situation.

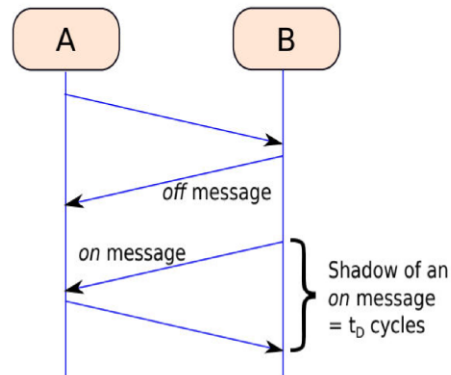


Figure 8.12: The shadow of an on message

Fig 8.12: ON message shadow

5.3 Message-Based (Circuit Switched) Flow Control

- Step 1: Compute route, send probe packet along path to reserve resources.
- Step 2: Probe sets up path (tells routers to reserve buffers + link access).
- Step 3: Destination sends ACK back to source.
- Step 4: Source begins sending message after receiving ACK.
- Latency = $3K + L/B - 1$ cycles (K =hops, L =packet length in flits, B =bandwidth).
- For long messages ($L/B \gg K$): overhead of probe+ACK is negligible.
- Advantage: routers need very few buffers; can use bufferless designs.
- Disadvantage: resources reserved longer than needed (during probe/ACK round trip).

- Subsequently, we require $K - 1$ cycles for the last byte to reach the destination node. We subtract 1 from K because we are already accounting for its transmission time in the expression L/B . The total time thus required is $3K + L/B - 1$ cycles.
- Let us visualize the process of transmitting the message in Figure 8.13. This diagram is known as a space-time diagram. The columns represent cycles, and the rows represent hops.
- Such diagrams are used to visualize the actions of a given flow control or message transmission protocol. We shall find them to be very useful while describing different flow control mechanisms.

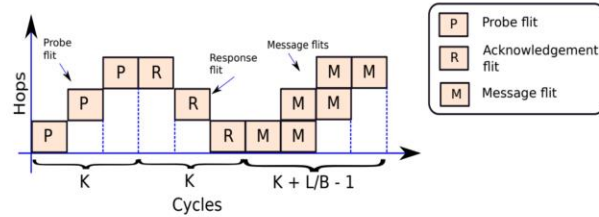


Figure 8.13: Space-time diagram for circuit switching. $K = 3$ cycles, $L = 2$ flits, $B = 1$ flit/cycle. Assume each packet contains a single flit. The y-axis is the hop count (measured from the source).

Fig 8.13: Space-time diagram for message-based flow control

6. Routing Algorithms

- Routing = choosing a path between source and destination nodes.
- Static routing: path decided in advance.
- Dynamic routing: path computed as message travels.
- Metrics: path length, deadlock freedom, livelock freedom.

The process of choosing a path between two nodes in an on-chip network is known as routing. Routing can primarily be of two types: static and dynamic. When the path between two nodes is known in advance, we refer to this method as static routing. In contrast, when the path is not fixed, and is decided as the message is travelling from the source to the destination node, we refer to this method as dynamic routing.

Fig: Routing classification

Definition 58 A livelock is defined as a general condition where the state of the system changes continuously; however, there is no long term progress. The classic example of a livelock is two people approaching each other in a corridor. Both of them try to cross each other; however, the moment one person moves to his left, the person on the other end moves to his right (and vice versa). Hence, they continue to face each other, and even though they are moving, they fail to make progress and reach their destinations!

Starvation is defined as a situation where a message is not able to make progress because routers chose to transmit other messages in its place for an indefinite period. Either it fails to get injected into the network or it gets stuck at an intermediate node because other messages are transmitted in its place.

Fig: Routing types overview

A deadlock is defined as a situation where multiple nodes try to send messages, yet none of them are successful because there is a circular wait. Assume that the nodes are numbered $V_1 \dots V_n$, and node V_i waits on node V_{i+1} for it to free some space such that it can send a message to it ($V_i \rightarrow V_{i+1}$). We also try to send a message from node V_n to V_1 . We thus have a circular wait of the form: $V_1 \rightarrow V_2 \dots \rightarrow V_n \rightarrow V_1$. It is possible to resolve such a deadlock only if we can remove some messages from the nodes and send them along different paths.

Fig: Routing algorithms detail

- Our main aim while trying to route a message between two points of the NoC is that we want the message to reach in the shortest possible time. Often we would like to maximise the throughput of the network, which means that we should be able to transport the maximum number of bytes per unit time between all pairs of transmitting sources and destinations.
- If there is no congestion in the network, then the shortest path between two nodes is also the path that takes the least amount of time. However, if we have congestion at the nodes, then the problem becomes complicated. We need to take the network congestion into account, and this creates the need to often take longer paths in terms of the number of links traversed.

Fig: Routing decision factors

6.1 Handling Starvation and Livelocks

- Starvation: use fair routers with priority schemes.
- Livelocks: add hop count field in head flit; increment at each router.
- Set threshold (e.g., $2 \times$ shortest path). After threshold: route only via shortest path.
- Alternative: give higher priority to packets with higher hop count.

6.2 Deadlocks & Dependence Graphs

- Deadlock: circular wait among flits — none can make progress.

- Consider a system of four nodes as shown in Figure 8.25. Assume we have a single virtual channel (VC) per physical channel.
- Here, packet P1 at node A is trying to use channels 1 and 2, P2 at node B is trying to use channels 2 and 3, P3 at C is trying to use channels 3 and 4, and P4 at D is trying to use channels 4 and 1.

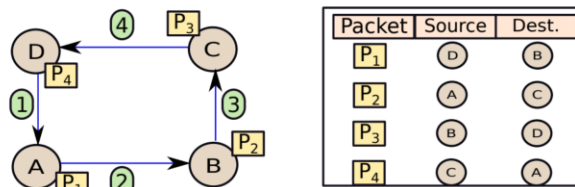


Figure 8.25: Deadlock in an NoC (single VC per channel)

Fig 8.25: Deadlock example with 4 nodes

- Resource Dependence Graph (RDG): nodes = agents (packets) + resources (channels).
- Hold edge: resource R $\rightarrow$ agent A (A holds R).
- Wait edge: agent A $\rightarrow$ resource R (A waits for R).

- Cycle in RDG = deadlock.

Architectures for Hardware Acceleration

Network on Chips: Routing: Dependence Graphs



- To model this situation and even more complex situations, we need to introduce a new theoretical tool called the Resource Dependence Graph (RDG) shown in Figure 8.26.
- Let us have two kinds of nodes: agents and resources. In this case an agent is a packet and the resource is a channel. We add an edge (hold edge) from resource R to agent A, if A holds R.
- Similarly, we add an edge from A to R (wait edge) if A is waiting for R to become free. Note that the hold and wait edges are in opposite directions.

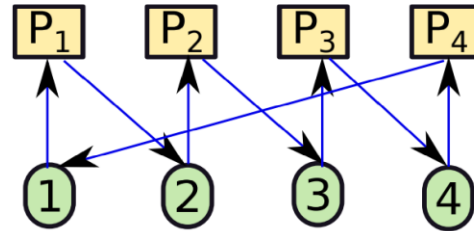


Figure 8.26: Resource dependence graph for Figure 8.25

Fig 8.26: Resource Dependence Graph

Architectures for Hardware Acceleration

Network on Chips: Routing: Dependence Graphs



- A resource dependence graph (RDG) for the scenario shown in Figure 8.25 is shown in Figure 8.26.
- Consider the case of packet P1. It holds channel 1, hence there is an arrow from channel 1 to P1. P1 waits for channel 2 to become free. Hence, there is an arrow from P1 to channel 2. The rest of the edges are added in a similar manner.

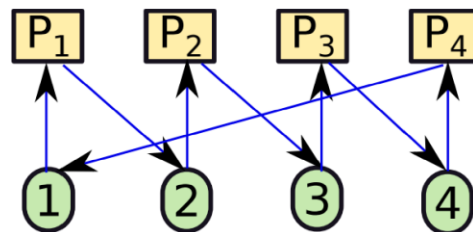


Figure 8.26: Resource dependence graph for Figure 8.25

Fig 8.27: Channel Dependence Graph (CDG)

- CDG: simplified RDG with only channels as nodes.
- Edge from C to C' if agent holding C waits for C'.
- Channels are unidirectional; buffer space at receiver node.
- No cycle in CDG = no deadlock.

6.3 Dimension-Ordered (X-Y) Routing

- Route from A(x1,y1) to B(x2,y2): first traverse X direction, then Y direction.
- Always yields shortest path. No livelock (no circular movement).
- Provably deadlock-free (CDG is acyclic).

- Limitation: fixed path per source-dest pair → no flexibility for congestion.
- Only allows 4 of 8 possible turns: E→N, E→S, W→N, W→S.
- Prohibits: N→E, N→W, S→E, S→W (no turns after Y-axis travel).

Architectures for Hardware Acceleration

Network on Chips: Routing: Dimension-Ordered Routing

- Let us consider the simplest class of deadlock-free routing protocols called dimension-ordered routing, the simplest of which is X-Y routing.
- Consider a mesh of nodes as shown in Figure 8.30. Each node has an x coordinate and a y coordinate. Let us describe the algorithm for routing a message from node A (x_1, y_1) to node B (x_2, y_2).

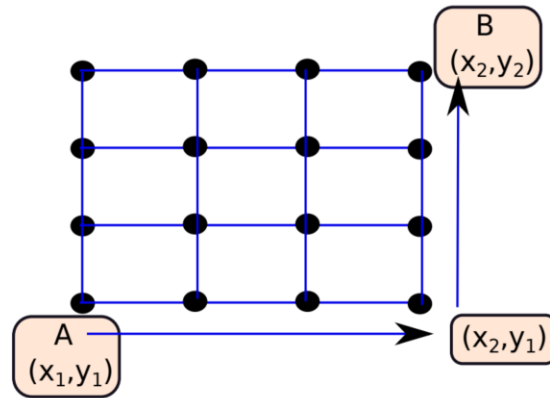


Figure 8.30: X-Y routing in a mesh

Fig 8.30: X-Y routing in a mesh

6.4 Oblivious Routing (Valiant's Algorithm)

- Step 1: Choose random pivot point P in network.
- Step 2: Route from source A to P using X-Y routing.
- Step 3: Route from P to destination B using X-Y routing.
- Advantage: randomizes routes, avoids hotspots, uniform traffic spread.
- Disadvantage: longer paths (pivot may be far from both source and dest).
- Trade-off: congestion avoidance vs. end-to-end latency.
- Deadlock-free (uses X-Y routing in each step).

Architectures for Hardware Acceleration

Network on Chips: Oblivious Routing

Let us thus describe the Valiant's algorithm. The algorithm is very simple; the steps are as follows: -

1. Assume we want to route a message from point A to point B. Choose a random point P in the mesh. Let us call it the *pivot point*.
2. First route the message from A to P using a provably deadlock-free algorithm such as X-Y routing.
3. Then route the message from P to B using a similar algorithm as used in the previous step.

6.5 Minimally Oblivious Routing

- Pivot point P restricted to vicinity of destination (not random).
- Reduces additional detour while maintaining some path diversity.
- Trade-off between congestion experienced and path length.
- Path Diversity = number of paths from source to destination.

6.6 Adaptive Routing & Turn Model

- Router directions: N, S, E, W (+ local port). 5 input ports, 5 output ports.
- 8 possible turns: $N \rightarrow E$, $N \rightarrow W$, $S \rightarrow E$, $S \rightarrow W$, $E \rightarrow N$, $E \rightarrow S$, $W \rightarrow N$, $W \rightarrow S$.
- Clockwise cycle turns: $E \rightarrow S$, $S \rightarrow W$, $W \rightarrow N$, $N \rightarrow E$.
- Anti-clockwise cycle turns: $W \rightarrow S$, $S \rightarrow E$, $E \rightarrow N$, $N \rightarrow W$.
- To prevent deadlock: eliminate ≥ 1 turn from each cycle.
- 16 possible routing algorithms allowing 6 of 8 turns.

Architectures for Hardware Acceleration

Network on Chips: Routing: Adaptive Routing



- A flit going north can take two possible turns: go west, or go east. Let us designate these turns as $N \rightarrow W$ and $N \rightarrow E$ respectively.
- Similarly, for all the other directions, there are two possible ways in which we can take turns. There are thus 8 possible turns that a message can take.
- Let us quickly look at the number of turns that are allowed in X-Y routing. Recall that when a flit is travelling along the y-axis, it cannot take a turn in the x direction.

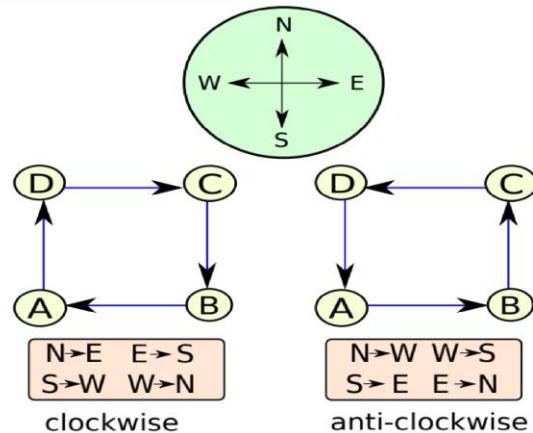


Figure 8.34: Basic turns in a turn graph

Fig 8.33: Router directions and turns

- The key learning here is that in a clockwise cycle, we shall definitely have 4 turns: $E \rightarrow S$, $S \rightarrow W$, $W \rightarrow N$, and $N \rightarrow E$.
- Similarly, in an anti-clockwise cycle, we shall also have these 4 turns: $W \rightarrow S$, $S \rightarrow E$, $E \rightarrow N$, and $N \rightarrow W$.
- From each cycle, if we can eliminate at least one turn, then we are sure that cycles will not form: clockwise or anti-clockwise.
- If cycles do not form in the TG, then as we have discussed earlier, there is no possibility of a deadlock.

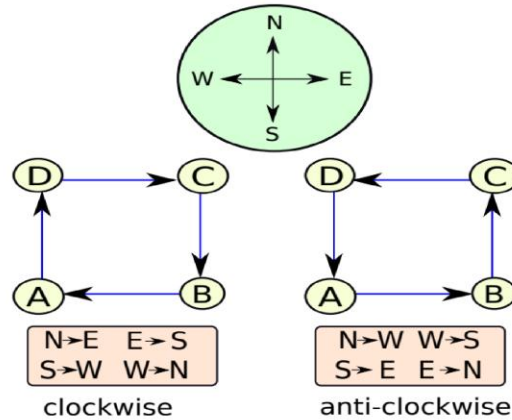
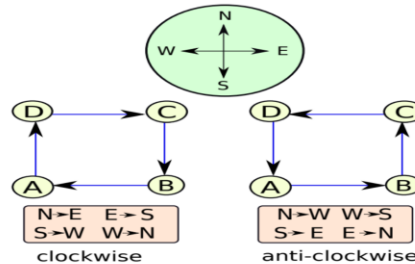


Figure 8.34: Basic turns in a turn graph

Fig: Clockwise and anti-clockwise cycles

- For each cycle, we need to prohibit one turn. We have four possible choices for each cycle (clockwise and anti-clockwise), and thus there are 16 possible choices overall.
- We can thus design 16 possible routing algorithms that allow 6 out of 8 turns. Let us review a few of them in Table 8.2.



8.34: Basic turns in a turn graph

Name	Turns disallowed	Allowed turns
West-first	$N \rightarrow W$, $S \rightarrow W$	
North-last	$N \rightarrow W$, $N \rightarrow E$	
Negative-first	$N \rightarrow W$, $E \rightarrow S$	

Table 8.2: Routing algorithms with 6 allowed turns

Table 8.2: Turn-model routing algorithms

- West-first: always go west first; prohibits $N \rightarrow W$, $S \rightarrow W$.
- North-last: always go north at end; prohibits relevant turns.
- Negative-first: go in negative directions (west/south) first.

7. Router Design

- Router = most important NoC component. Pipelined for high throughput.
- Pipeline stages: Input Buffering $\rightarrow$ Route Computation $\rightarrow$ VC Allocation $\rightarrow$ Switch Allocation $\rightarrow$ Switch Traversal.

7.1 Input Buffering

- Flits stored in buffer queues at input ports.
- Packet/circuit switched: single buffer queue per physical channel.
- Flit-based with VCs: multiple queues per physical channel (one per VC).
- Each flit must know its VC ID to be queued correctly.
- Two implementations:
 1. Separate queues (SRAM): simple but limits storage (can't share space across VCs).
 2. Shared buffer: single large flit array with head/tail pointers per VC. More flexible.

Architectures for Hardware Acceleration

Network on Chips: Router Design: Input buffering



- If storage space is our primary concern, we can use a shared buffer based approach as shown in Figure 8.37.
- In this case, we have a single, large array of flits. Different VCs are assigned to different non-overlapping regions of this array.
- Conventionally, the two ends of a queue are referred to as the head and tail respectively. The only state that we need to maintain in this case for each VC is the id of its corresponding head and tail pointers.
- Whenever we insert or remove a flit from a queue, we need to update these pointers.
- The advantage of this design is that the storage structure is more flexible. If a given VC needs a lot of space at any given point in time, and the rest of the VCs are unutilized, then it is possible to accommodate this requirement using this design.

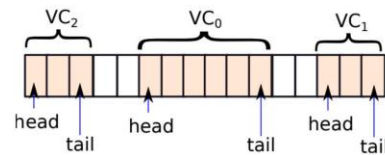


Figure 8.37: All the VCs using a single shared buffer

Fig 8.37: Shared buffer design for VCs

7.2 Route Computation

- Done only for head flit; rest follow same route.
- Head flit contains destination node ID.
- RC engine computes next router direction (N/S/E/W).
- Two methods:
 - Source Routing: entire route embedded in head flit as queue of directions.
 - Each router removes head of queue and sends packet that direction.
 - Static, no congestion awareness, overhead in head flit modification.
 - Node Table Routing: each router has a table mapping (direction, destination) → next direction.
 - Supports dynamic routing, congestion-aware decisions.
 - Route decisions based on: distance to dest, free VCs, buffer occupancy.

7.3 Virtual Channel (VC) Allocation

- After route computed, allocate a VC on the outgoing physical channel.
- Multiple packets from different channels may compete for same VCs.
- VC allocation written to head flit; all flits in packet use same VC.

- The design of the router as described till this point is shown in Figure 8.39. We are assuming two VCs per physical channel.
- In the first stage, we store the flit in the input buffers corresponding to the VC. Then in the next stage we send the data to the route computation unit via a pipeline register.
- After we have computed the route, we allocate a VC in the next cycle. We do not have a pipeline register at the end of this stage because we assume that the allotted VC is written to the head flit of the packet.

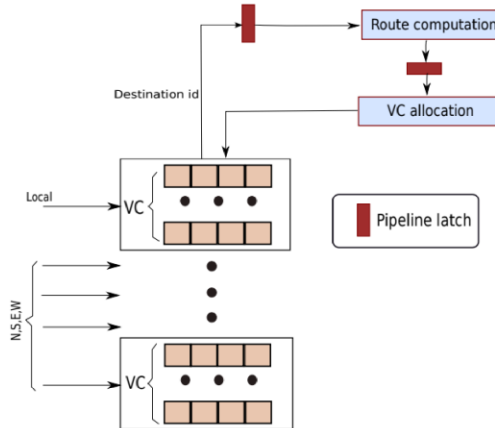


Figure 8.39: Router with the first three stages: input buffering, route computation, and VC allocation

Fig 8.39: Router pipeline with VC allocation

7.4 Switch Allocation

- Maps input VCs to output physical channels through a switch.
- Example: 2 VCs/channel $\times$ 5 channels = 10 input VCs, 5 output channels.
- Only one flit per outgoing channel at a time.
- Must be fair to avoid starvation.
- Optimization strategies:
 - Priority-based: route high-priority packets first.
 - Body/tail preference: quickly clear VCs after head flit sent.
 - Smallest packet first: minimizes average end-to-end latency (provably optimal).

- We thus have 10 input VCs. It is possible that all of these VCs have flits in them, which are waiting to be transmitted via outgoing links.
- In this case, we have 5 outgoing links: local, N, S, E, and W. Thus, the problem of resource allocation is as follows. We have 10 input channels and 5 output channels.
- To effect a packet transmission, we need to map an input to an output. Such a hardware structure is known as a switch. The 10×5 switch for our router is shown in Figure 8.40.

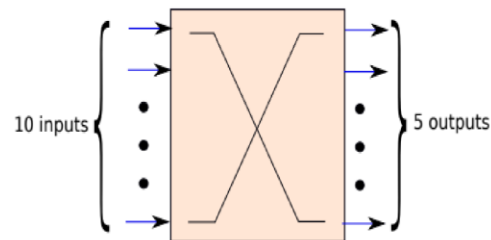


Figure 8.40: 10×5 switch

Fig 8.40: 10×5 switch structure

7.5 Switch Traversal & Crossbar

- Switch = array of pass transistors (rows=inputs, columns=outputs).
- Connection made by enabling transistors between horizontal/vertical wires.
- Crossbar: 2D matrix switch. Simple but expensive for large sizes.
- No output buffering in modern routers (2020+): flit goes directly to outgoing link.
- Crossbar metrics:
 - - Area cost = $N \times M$
 - - Latency cost = $N + M$
 - - For 10x5 router: area cost=50, latency cost=15.

Architectures for Hardware Acceleration

Network on Chips: Router Design: Switch Traversal

- We normally design a switch as an array of pass transistors, where the number of rows is equal to the number of input ports, and the number of columns is equal to the number of output ports.
- Here, each input port corresponds to an input VC, and each output port corresponds to an outgoing link (physical channel).
- We connect an input to an output by enabling the transistors between the horizontal and vertical wires.
- This creates a connection and data can thus be transferred from the input to the corresponding output.



Fig 8.41: Crossbar switch design

7.6 Optimized Switch Designs

- Monolithic crossbar too expensive. Hierarchical designs used:
- Design 1 - Full Hierarchical:
 - Five 2x1 switches + one 5x5 switch. Area=35, Latency=13.
 - Problem: can't send from both VCs of same channel simultaneously.
- Design 2 - Partial Hierarchical:
 - Two 5x2 switches + one 4x5 switch. Area=40, Latency=16.
 - Better flexibility, assumes max 4 simultaneous outgoing packets.

Architectures for Hardware Acceleration

Network on Chips: Router Design: Optimised Designs of Crossbar Switches



- There are several strategies that could be adopted for creating a more efficient design. The first is that we use a hierarchical approach.
- Out of the 10 VCs, we can first choose 5: one for each incoming link. Then we can have a 5×5 switch. This hierarchical strategy is shown in Figure 8.42.
- We first have a set of five 2×1 switches, and then we have a single large 5×5 switch. The area cost is the sum of 5 times 2×1 , and 25 (5×5), which comes to 35, and the latency cost is $2 + 1 + 5 + 5$, which is 13.
- The main problem with this design is that it does not allow both the VCs of a single physical channel to send flits simultaneously when they are destined for different outgoing channels.

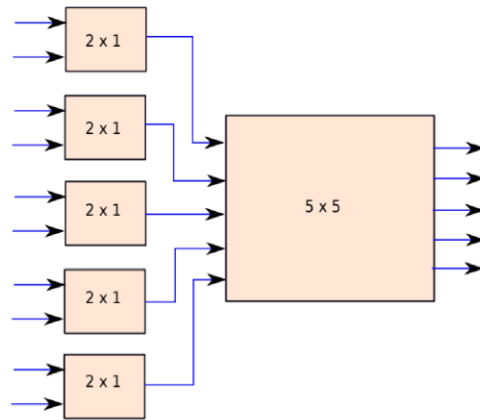


Figure 8.42: Hierarchical switch with 10 inputs and 5 outputs

Fig 8.42: Hierarchical switch design

Architectures for Hardware Acceleration

Network on Chips: Router Design: Optimised Designs of Crossbar Switches



- Let us try to solve the problem with the hierarchical switch by creating a different kind of grouping.
- Let us again create a 2-level hierarchy, where the first level has two 5×2 switches. The insight here is that we do not always have enough traffic to keep all the 5 links busy.
- We can safely assume that for an overwhelming majority of time, a maximum of 4 packets are simultaneously flowing through the outgoing links.

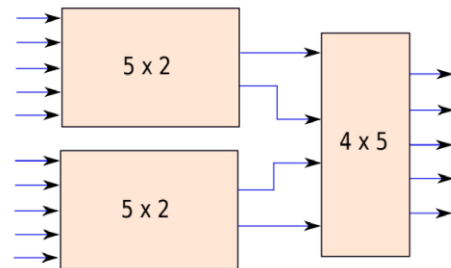


Figure 8.43: Hierarchical switch with wider internal switches

Design		Area cost	Latency cost
Level 1	Level 2		
10×5	-	50	15
Five 2×1	5×5	35	13
Two 5×2	One 4×5	40	16

Table 8.3: Area and latency costs of different switch designs

Fig 8.43: Alternative hierarchical design

7.7 Dimension Slicing

- Exploits X-Y routing asymmetry: X-packets can turn, Y-packets go straight only.
- Switch 1 (6×3): inputs = E, W, local (2 VCs each); outputs = E, W, link to Switch 2.
- Switch 2 (5×3): inputs = 1 from Switch 1 + N, S (2 VCs each); outputs = N, S, local.
- Reduces switch complexity compared to monolithic design.

- First consider a switch that has three inputs: E, W, and local. Packets can be sent along any of its outgoing links.
- This switch has three outgoing links: E, W, and a link to another switch. It is thus a 6x3 switch as shown in Figure 8.44.
- Note that we are assuming two VCs for each physical channel.

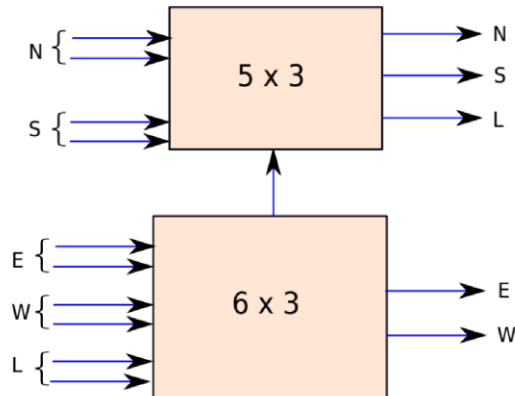


Figure 8.44: Dimension-sliced switch

Fig 8.44: Dimension-sliced switch design

8. Allocators and Arbiters

- Problem: N requests, M resources. Find mapping (matching) between them.
- Conditions: each request mapped to ≤ 1 resource, each resource to ≤ 1 request.
- Maximum matching: map maximum possible requests. Hard in hardware.
- Maximal matching: no more mappings possible (easier, used in practice).

8.1 Round Robin Arbiter

- Resource distributed to agents in circular order: $1 \rightarrow 2 \rightarrow \dots \rightarrow N \rightarrow 1$.
- N request lines + N grant lines + N priority lines.
- AND gate per agent: Grant = Request AND Priority.
- NOR gate G1: outputs 1 when no grants asserted.
- D flip-flop stores priority for next cycle based on last grant.
- Drawback: agents not needing resource still block next agent's turn.
- Fix: timeout logic to auto-assert grant for inactive high-priority agents.

Architectures for Hardware Acceleration

Network on Chips: Router Design: Round Robin Arbiter



- In this case, the resource is distributed among the requesting agents in a round robin fashion. This means that at the outset the resource is given to requesting agent 1, then to agent 2, till we reach agent N.
- The assumption is that the total number of agents in the system is equal to N. After agent N finishes using the resource, we again come back to agent 1, and continue in the same fashion.

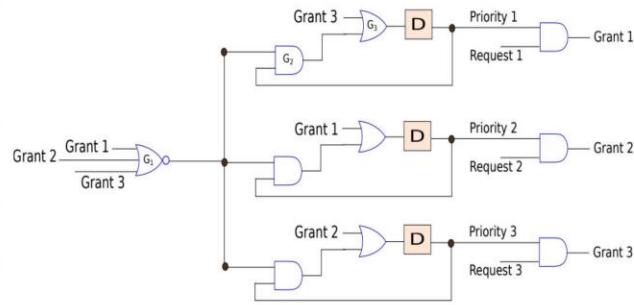


Figure 8.46: A simple round robin arbiter

Fig 8.46: Round Robin Arbiter circuit

Architectures for Hardware Acceleration

Network on Chips: Router Design: Round Robin Arbiter



- Figure 8.46 shows a simple round robin arbiter. We assume that there are N request lines and N grant lines. Each request line is connected to a requesting agent.
- If the agent needs to place a request, it raises the voltage of its request line to a logical 1.
- On similar lines, there are N grant lines. Whenever a request is granted to the i th agent, the i th grant line is set to 1. The rest of the grant lines are set to 0.

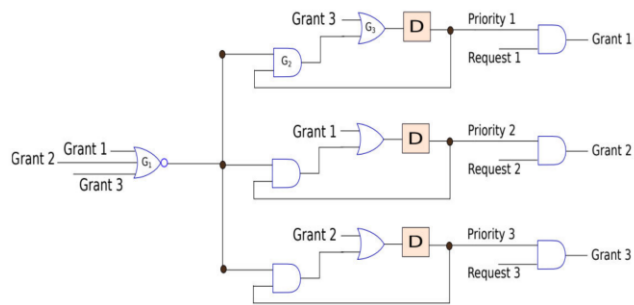


Figure 8.46: A simple round robin arbiter

Fig: Round Robin Arbiter detail

8.2 Matrix Arbiter

- $N \times N$ Boolean matrix W . Properties:
 - - $W[i,i] = 0$ (diagonal)
 - - $W[i,j] = \text{NOT } W[j,i]$ for $i \neq j$
- Agent i gets resource when all non-diagonal entries in row i are 1.
- After grant: reset row to 0s, set column to 1s (lowest priority).
- More flexible than round robin; skips uninterested agents.
- If agent has no request when eligible: auto-relegate to lowest priority.

Architectures for Hardware Acceleration

Network on Chips: Router Design: Matrix Arbiter



This means that the diagonal entries have null values. Secondly, $\mathcal{W}[i, j]$ and $\mathcal{W}[j, i]$ have dissimilar values when $i \neq j$. In this matrix, $\mathcal{W}[i, j] = 1$ means that agent i has a priority over agent j . This implies the following:

If all the entries in the i^{th} row are 1 (other than the diagonal element), then the i^{th} agent has the highest priority. It can be granted the resource.

The basic steps in the arbitration algorithm are as follows.

1. If a given agent is not interested then it sets all the entries in its row to 0, and all the entries in its column to 1 (other than diagonal elements).
2. In every cycle, the request is granted to the agent, which has ones in all of the entries in its row other than the diagonal element.
3. Once the i^{th} agent is done servicing its request, it sets all the entries in its row to 0, and sets all the entries in its column to 1 (other than the diagonal elements). This means that it relegates itself to the lowest priority.

Fig 8.47: Matrix Arbiter example

Architectures for Hardware Acceleration

Network on Chips: Router Design: Matrix Arbiter



- Figure 8.47 shows an example. In this case, agent 1 is assigned the resource first because all the non-diagonal entries in its row are 1.
- Subsequently, it resets all the entries in row 1, and sets all the entries in column 1. Since agent 2 is not interested in acquiring the resource, the next agent that should be granted the resource is agent 3 (all the entries in its row are 1).
- After agent 3 is done, it follows the same procedure. The next agent that should be granted the resource is agent 4. After agent 4 is done, it sets all the entries in column 4 to 1.

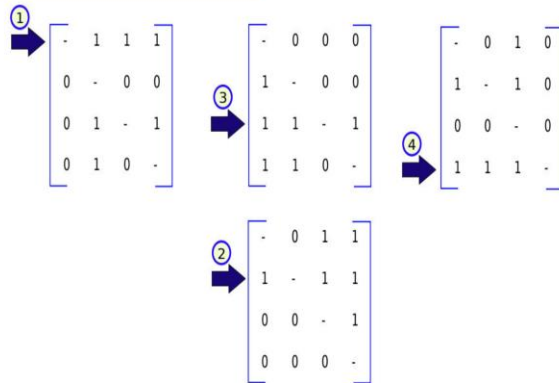


Figure 8.47: Matrix arbiter. The '-' symbol represents a null value.

Fig: Matrix Arbiter state transitions

8.3 Separable Allocator

- For N agents, M resources. Two columns of arbiters.
- Column 1: one arbiter per agent — chooses one request from each agent.
- Column 2: one arbiter per resource — chooses one agent per resource.
- Can lead to sub-optimal allocations (e.g., all agents' chosen requests target same resource).

- The separable allocator has two columns of arbiters. An arbiter in the first column chooses one request from each agent.
- Recall that we cannot allocate two requests issued by the same agent at the same time. We need to choose one of the agent's requests.
- This is done by the 3×3 arbiters in the first column. Since we have three agents, we have three arbiters.
- Consider the first arbiter in the first column. It has three inputs: R11, R12, and R13. There are three grant lines as outputs – one corresponding to each request.

Fig 8.48: Separable Allocator

8.4 Wavefront Allocator

- $N \times N$ matrix (rows=agents, columns=resources). Connected like a torus.
- Diagonal elements get row token + column token initially.
- Row tokens move left each round; column tokens move down (with wraparound).
- Cell with both tokens + interested $\rightarrow$ consumes both tokens = allocation.
- Consumed tokens block further allocations in that row and column.
- Wavefront of cells with 2 tokens propagates diagonally.
- Properties: wavefront size = N ; never 2 cells in same row/column.
- More optimal than separable: simultaneous global allocation.

- In the matrix W (shown pictorially in Figure 8.49), we start out with giving one row token and one column token to each of the diagonal elements. If agent i is interested in acquiring resource j then we mark the cell $W[i, j]$.
- It is shown with a shaded colour in Figure 8.49. The matrix is connected like a torus, where neighboring cells in a row and column are connected, and there is a connection between the first cell of a row and the last cell of the row (likewise in a column).

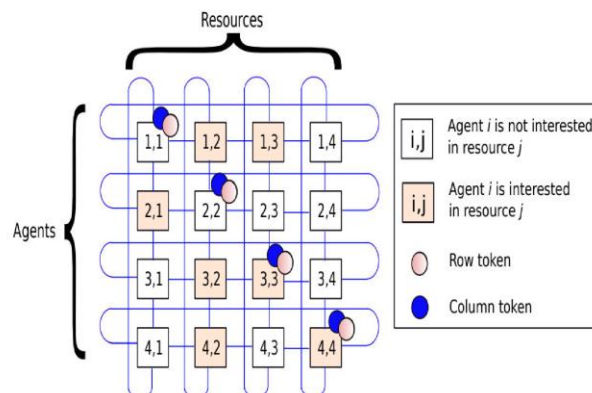


Figure 8.49: The matrix W with agents, resources, and connections between neighbouring cells

Fig 8.49: Wavefront Allocator matrix

- The algorithm consists of multiple rounds. In each round, each cell executes the following steps.
 - If a given cell in the matrix has a row token and a column token, and is interested in acquiring a resource, then it is allotted that resource. Cell $\mathcal{W}[i, j]$ is interested in acquiring a resource if agent i is interested in getting access to resource j . This cell *consumes* both the tokens, and removes them from the system.
 - If a cell is not interested in a resource, and it has a token, then it sends the token to a neighbouring cell such that it can be processed in the next round. If it has a row token it sends it to the cell on the left, and if it has a column token then it sends it to the cell that is just below it.
 - If a token reaches an edge of the matrix, it traverses the long edges to wrap around the matrix and start from the other end of the matrix (same row or same column) in the next round.

Fig 8.50: Token movement across rounds

- For example, the cells (2,1) and (3,2) are not free in the second round. Also observe that they have a single token each. The remaining tokens propagate as per the rules that we have defined.
- Again in round 3, the cell (3,1) consumes the two tokens that it gets. Finally, we have 4 tokens left in round 4. Let us look at the cells that are free in each round. We observe that our earlier observation still holds.
- All the free cells either have two tokens or do not have any token. We never have a situation where a free cell has a single token.
- This is not possible because for a cell to have a single token, one of the tokens in its row or column must have been consumed. This means that the cell is not free anymore.

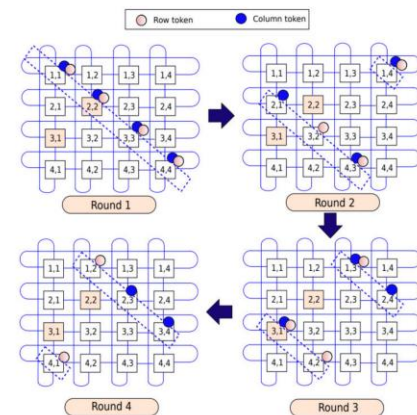


Figure 8.51: The movement of tokens in different rounds when two cells are interested in acquiring a resource.

Fig 8.51: Wavefront with active requests

9. Non-Uniform Cache Architectures (NUCA)

- As cache sizes grew beyond MBs, delay in routing to correct subarray became bottleneck.
- NoC-based bank design allows parallel accesses to different banks.
- 8x8 mesh (32 cores + 32 banks): corner-to-corner = 15 routers, 14 links.
- Router delay: 3-5 cycles each. NoC delays vary 5 to 50 cycles across banks.
- Key idea: place data closer to requesting cores to reduce latency.

9.1 Static NUCA (S-NUCA)

- Cache blocks allotted to banks statically using bits from block address.
- Example: 32-bit address, 64-byte block, 256 sets/bank, 32 banks.
- - 6 bits for byte offset, 8 bits for set, 5 bits for bank, 13 bits tag.
- Two addressing schemes:
- S_{LSB} : use 5 LSB bits of block address for bank → more uniform bank selection.
- $S_{\text{no-LSB}}$: use bits 9-13 for bank → more uniform set access within bank.
- Trade-off: bank conflicts vs. set conflicts (depends on workload, needs simulation).
- Lower bits have more randomness (spatial locality → lower bits vary more).

Architectures for Hardware Acceleration

Network on Chips: Static NUCA(S-NUCA)



- Let us answer this question by considering multiple designs. Consider the two alternatives shown in Figure 8.58.
- We show two schemes: S_{LSB} and $S_{\text{no-LSB}}$. In the S_{LSB} scheme, we choose the 5 LSB bits (bits 1 to 5) of the block address to uniquely address each bank. Bits 6-13 are used to address the set.
- In the $S_{\text{no-LSB}}$ scheme, we use the 8 LSB bits (bits 1 to 8) of the block to address the set, and then we use bits 9-13 to address the bank. These are the two most popular schemes.

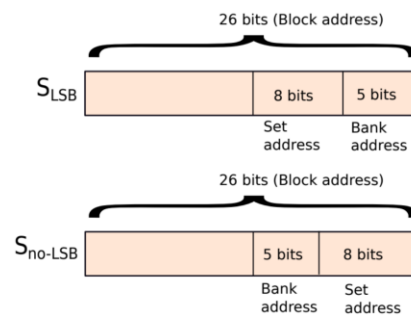


Figure 8.58: Two methods to address a bank and a set in the S-NUCA protocol

Fig 8.58: S-NUCA addressing schemes (S_{LSB} vs $S_{\text{no-LSB}}$)

9.2 Dynamic NUCA (D-NUCA)

- Blocks can migrate between banks at runtime based on access patterns.
- Banks divided into columns called bank sets.
- A block can be in any bank within its bank set (no replication).
- Home bank: closest bank in the bank set to the requesting core.
- Request goes to home bank first.

Architectures for Hardware Acceleration

Network on Chips: Dynamic NUCA(D-NUCA)



- The main disadvantage of an S-NUCA protocol is that we are not taking the usage of blocks into account.
- We should ideally place a frequently used block close to the requesting core. This is not happening. Hence, let us propose the D-NUCA protocol that places blocks close to the requesting cores and migrates blocks based on the access pattern.
- The key idea is to divide the set of banks into columns as shown in Figure 8.59.
- We call each such column a bank set. A block can be present within any bank of the bank set. There is no replication though.

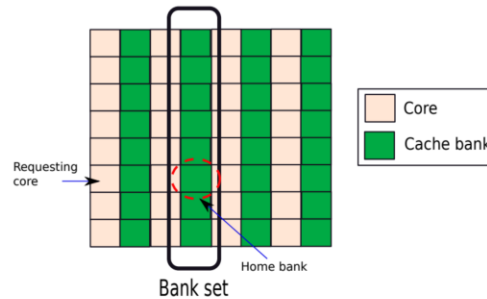


Figure 8.59: Bank sets in D-NUCA. The cores and cache banks are organised in columns.

Fig 8.59: D-NUCA bank sets and home bank

Architectures for Hardware Acceleration

Network on Chips: Dynamic NUCA(D-NUCA)



- In Figure 8.59 we have 4 bank sets. We can use 2 bits from the block address to determine the bank set. This is where we need to do something to ensure that we are able to achieve our objective of placing data close to the requesting cores.
- Let us designate one of the banks within the bank set as the home bank. There is no hard and fast rule regarding how we designate the home bank.
- However, most research proposals in this area designate the bank in the bank set that is the closest to the requesting core (shortest routing path) as the home bank.
- This is shown in Figure 8.59. The requesting core sends the memory request to the home bank first; it has a separate home bank in each bank set.

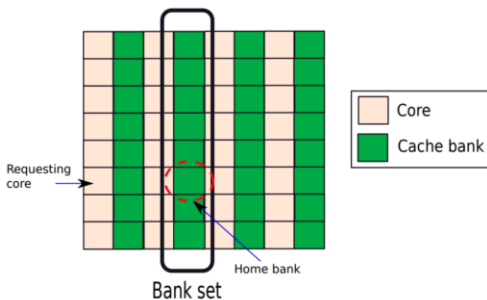


Figure 8.59: Bank sets in D-NUCA. The cores and cache banks are organised in columns.

Fig: D-NUCA structure detail

9.2.1 Search Policies

- If miss in home bank, search rest of bank set using one of three policies:
 1. Sequential: search banks one by one from home bank outward.
 2. Two-way: search in both directions from home bank simultaneously.
 3. Broadcast: send request to all banks in bank set at once.
- Trade-off: messages sent vs. banks searched vs. overall latency.
- Broadcast increases bank utilization and contention — not always better.
- If block not found anywhere: signal miss → request goes to lower memory level.

Architectures for Hardware Acceleration

Network on Chips: Dynamic NUCA(D-NUCA): Search Policies



- Once the request reaches the home bank, we need to search for the block. We first search within the home bank. If there is a hit, then we send the value back to the requesting core.
- If there is a miss, then we need to search the rest of the banks within the bank set. In this case each bank set is a column – a linear sequence of banks.
- We need to search the rest of the banks according to a given search policy.
- There are three types of commonly used search policies as shown in Figure 8.60. Note that in this scheme, we never replicate a block across banks; every block has a single location.

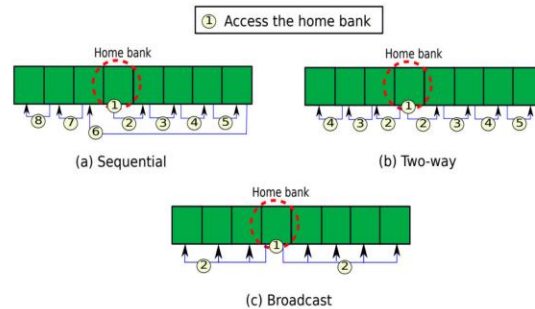


Figure 8.60: Different search policies within a bank set

Fig 8.60: D-NUCA search policies

Architectures for Hardware Acceleration

Network on Chips: Dynamic NUCA(D-NUCA): Search Policies



- The search policies are as follows:

Sequential We search all the banks in one direction and go till the end. If we do not find the block, then we start searching in the other direction till we reach the end.

Two-Way In this scheme, we send two parallel messages in both the directions. If the block is present in the bank set, then it will be found in only one bank. As compared to the *Sequential* scheme, we are expected to get a response sooner.

Broadcast In this scheme, we broadcast a message to all the banks. On an average, we have a lot of additional bank accesses. However, this is also the fastest scheme if we do not take the contention within banks and the NoC into account. We do not have to waste time searching banks that do not contain the block.

Fig: Search policy comparison

9.2.2 Migration

- On hit (not at home bank): migrate block one hop closer to home bank.
- Repeated accesses → block eventually reaches home bank.
- On eviction from a bank: two choices:
 1. Write to lower level immediately.
 2. Move to bank further from home bank (evict from cache only at last bank).
- Option 2: block lingers longer (good if reused; bad if dead block).
- Migration reduces cache access latency significantly over time.
- Method of choice in most NUCA designs.

Architectures for Hardware Acceleration

Network on Chips: Dynamic NUCA(D-NUCA): Migration



- Otherwise, we migrate the block towards the home bank. Figure 8.61 shows an example. Initially, we have a hit in bank A, which is not the home bank.
- We move the block one hop towards the home bank. This means that the next time the same core requests for the same block, the request has to travel one hop less.
- Ultimately, the block will migrate to the home bank, and remain there assuming that there are no conflicting requests from other cores with different home banks.

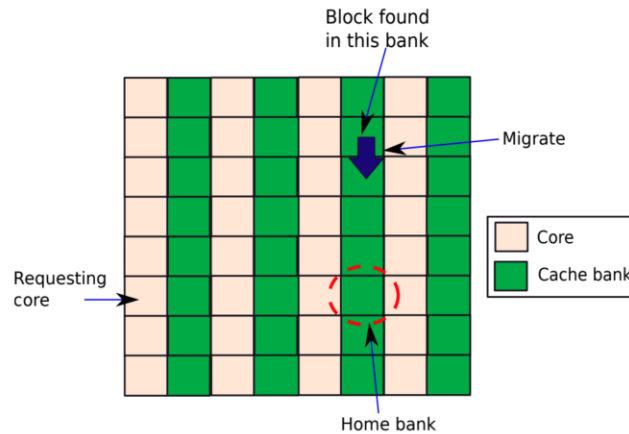


Figure 8.61: Block migration in a D-NUCA cache

Fig 8.61: Block migration towards home bank

Architectures for Hardware Acceleration

Network on Chips: Dynamic NUCA(D-NUCA): Migration



- By migrating blocks towards the home bank, we are in effect reducing the cache access latency for that block. We are also reducing the number of bank accesses in the Sequential and Two-way search policies.
- When there is an eviction from a bank, we have two choices. Either we write the block to the lower level, or we write it to a bank that is further away from the home bank.
- In the latter scheme, a block is finally evicted from the cache and written to the lower level when it is evicted from the last bank (rim of the chip) in the bank set.

Fig: Migration example detail

10. NoC Performance Evaluation Metrics

10.1 Latency

- Time (in cycles) for a packet to go from source to destination.
- Affected by route choice and network congestion.
- Aggregation: mean latency, variance/mean ratio, mean latency per hop.
- Mean latency per hop indicates degree of congestion (independent of distance).

10.2 Throughput

- Number of bytes transferred per unit time throughout network.
- Measured by simulating network for many cycles and computing mean.
- Throughput $\neq$ Bandwidth (bandwidth = theoretical max; throughput = observed).
- Indicates overall efficiency of data transfer.

10.3 Power

- Minimize energy consumption by:
 - Reducing router size
 - Powering down unused router parts
 - Minimizing route lengths

10.4 Area & Routing Complexity

- NoC needs on-chip space for links (wires) and routers.
- Must ensure existing connections don't get lengthened.
- Requires sophisticated wire placement algorithms.
- Minimize router area to leave space for cores and caches.

Key Formulas Summary

- Mesh diameter ($N \times N$): $2N - 2$
- Torus diameter ($N \times N$): N
- Hypercube (N nodes): neighbors = $\log_2(N)$, diameter = $\log_2(N)$
- Butterfly: links = $N + N \cdot \log_2(N)$, diameter = $\log_2(N) + 1$
- Clos: total I/O = nr , strictly non-blocking when $m \geq 2n - 1$
- Credit round trip delay: $t_D = t_f + t_{ph} + 2 \cdot t_{pr}$
- Min buffers (credit): t_D / t_f
- N_{OFF} threshold (on-off): $\geq t_D / t_f$
- Message-based latency: $3K + L/B - 1$
- Crossbar area cost: $N \times M$, latency cost: $N + M$